

On this page

Blue-Green deployments

The recommended approach for zero downtime deployments is a technique known as blue-green deployment.

Blue-green deployment uses two almost identical clusters in the following way:

- At the start, one of the clusters is in use. Let's call it the blue environment.
- The other one is on standby, created from a new release. Let's call this one the green environment.
- When the green cluster is deployed, traffic is redirect to this cluster and deployment becomes the stand-by environment. It is available in case of a necessary rollback. We recommended this step to be manual so that every validation is done prior to changing traffic to the new cluster.



Prior to a Blue-Green deployment RabbitMQ should have colored vhosts set up (`pulseblue` and `pulsegreen`)

Using a control-node as Blue-Green deployment orchestrator¹

When SSH connection is available between the control-node and service machines (Pulse, RabbitMQ, etc) it can orchestrate the entire deployment, excluding only validation and switching load balancer tasks.

The first step is to federate the necessary queues, which can be done as follows:

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i project-<env> \
rabbitmq-setup-blue-green-federation.yml
```

Copy

Note: the above playbook should only run on a single RabbitMQ node.

Pulse machines are installed using the current production opposite color and by copying its metadata. This can be achieved by using Ansible variables `deployment_color=<opposite_color>` and `keep_metadata_between_deployments=true`:

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
pulse.yml
-e deployment_color=<opposite_color>
-e keep_metadata_between_deployments=true
```

Copy

After Pulse installation ends and the new cluster is validated, load balancers can be changed to point to the new cluster.

Federate queues can also be disabled by running the following playbook:

```
python3.6 ansible-playbook \
-i project-default \
```

```
-i project-hsbc \
-i gcp \
-i project-<env> \
rabbitmq-disable-blue-green-federation.yml
```

Copy

Note: the above playbook should only run on a single RabbitMQ node.

Temporary solution for joint session



This approach is currently under discussion and may not represent the final solution

When SSH connection between the orchestrator (control-node) and Pulse images is not available, the deployment can be achieved by offloading the steps that are prone to race conditions to a different machine(s).

The pipeline is defined as follows:

1. Deployment color is set to the current production environment opposite color
2. Pulse images are created with the latest Feedzai release
3. RabbitMQ queues replication (federation) is set up to allow new cluster to receive updates from the production cluster. Refer to [RabbitMQ federation](#) for details
4. A machine is created to clean up the existing metadata for colored cluster and to copy the metadata from the production cluster
5. Pulse machines are configured but ignore metadata and patches set up
6. A machine is create (can be the same as step 4) to apply patches and publish the application
7. After validating the new cluster, load balancers are updated to redirect traffic to the new cluster
8. RabbitMQ replication queues are removed
9. Old cluster may be kept for a period of time to be later removed. This ensures production cluster can be swapped to the previous stable cluster in case anything is wrong

Comparing to [Using a control-node as a Blue-Green deployment orchestrator](#), steps 4 to 6 split up existing Ansible roles while running from different machines.

The machine used in *Steps 3 and 8* to create and remove replication queues should have `rabbitmqctl` installed. This can be done by running the following playbook:

```
python3.6 ansible-playbook \
--connection=local \
-i project-default \
-i project-hsbc \
-i gcp \
-i project-<env> \
thirdparty.yml
--tags=rabbitmq.install,rabbitmq.440
```

Copy

Note: this machine should have TCP connection to RabbitMQ instances on ports `4369` and `25672` .

After the machine is provisioned replication queues can be created/destroyed by running `rabbitmq-setup-blue-green-federation.yml` and `rabbitmq-disable-blue-green-federation.yml` respectively.

```
python3.6 ansible-playbook \
--connection=local \
-i project-default \
-i project-hsbc \
-i gcp \
-i project-<env> \
rabbitmq-setup-blue-green-federation.yml # or rabbitmq-disable-blue-green-federation.yml to
destroy replication queues in Step 8
```

Copy

In *Step 4* only tags responsible for extracting and setting up the required components to run metadata clean up and copy current application metadata should be included. It is also required to set `keep_metadata_between_deployments=true` to enable metadata copy from existing cluster to the new cluster.

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
pulse.yml
-e deployment_color=<opposite_color>
-e keep_metadata_between_deployments=true
--tags pulse.050,pulse.060,pulse.110,pulse.120,pulse.210,pulse.220,pulse.415,pulse.425,pulse.426,pulse.440,pulse.455,pulse.710,pulse.730,pulse.731,pulse.750
```

Copy

In *Step 5* all tags should be included except tags that are responsible for metadata clean up, application patches and publishing. The first were already executed in the previous step and the latter will be executed afterwards.

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
pulse.yml
-e deployment_color=<opposite_color>
--tags=pulse.configure
--skip-tags pulse.730,pulse.731,pulse.750,pulse.932,pulse.940,pulse.941,pulse.970
```

Copy

In *Step 6* included tags depend whether it is running in the same machine as *Step 4* or in a new machine. In case it is running in the same machine the playbook can run including only tags `pulse.940` , `pulse.941` , and `pulse.970` .

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
pulse.yml
-e deployment_color=<opposite_color>
--tags pulse.940,pulse.941,pulse.970
```

Copy

When running in a new machine tags that set up the machine to run patches also need to be included. These are the same as in *Step 4* excluding tags `pulse.730` , `pulse.731` and `pulse.750` .

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i <region>/project-<env> \
-i <region>/project-<env>/<portfolio> \
pulse.yml
-e deployment_color=<opposite_color>
--tags pulse.050,pulse.060,pulse.110,pulse.120,pulse.210,pulse.220,pulse.415,pulse.425,pulse.426,pulse.440,pulse.455,pulse.710,pulse.940,pulse.941,pulse.970
```

Copy



Pulse Data Science deployments **should not** include `pulse.455` since this task is only required for Pulse Runtime deployments. Additionally, Pulse Data Science deployments should override the `apps_enabled` Pulse ansible variable to be empty: `apps_enabled: {}`



Take into account that with the introduction of the global rollout, images need to be created with dependencies for the applications that are to be deployed.

This applies to Pulse (RT), Case Manager and helper images for both components and can be controlled by the `apps_enabled` variable.

For example, in a market that should have `Outbound` and `Inbound` , all images should be generated with:

```
apps_enabled:  
  - <market>_outbound_payments  
  - <market>_reference_data  
  - <market>_inbound_payments
```

Copy